

Reviewer Pack — Minimal Rank of Quasi-Quadratic Forms vs AC0 Parity Circuit ...

Entry 6cd213d406ca · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 09:55:21 UTC

Minimal Rank of Quasi-Quadratic Forms vs AC0 Parity Circuit Size

Entry ID: 6cd213d406ca **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 09:55:21 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quasi-Quadratic Forms) **Field B** (complexity object): Complexity Theory: AC0 Parity Complexity

Statement:

[‘For any explicit boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, let Q_f be the quasi-quadratic form associated with the function such that its rank is bounded by $R(Q_f) = \Theta(n^{3/2})$. Then, the AC0 parity circuit size for f is lower-bounded by $\Omega(2^{R(Q_f)})$.’, ‘Equivalently, there exists an absolute constant $c > 0$ such that for all boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$, if $R(Q_f) = \Theta(n^{3/2})$, then the AC0 parity circuit size of f is at least $\Omega(2^{R(Q_f)}) / 2^{(c \cdot n^{3/4})}$.’, ‘This lower bound holds for all functions where the minimal polynomial of the associated quadratic form has a degree less than or equal to n .’]

Rationale (proposer’s reasoning):

[‘Quasi-quadratic forms provide a bridge between algebraic geometry and boolean functions, potentially capturing non-linear aspects that are not easily represented by linear forms.’, ‘This conjecture suggests that the complexity of parity computation in AC0 circuits could be deeply related to the geometric properties of these forms.’, ‘If true, this would offer new insights into the power of AC0 circuits and their limitations, possibly leading to a proof of $\text{NEXP} \neq \text{ACC}^0$.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 117b423743e655d6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all 30 random seeds, the AC0 parity circuit size exceeds $\Omega(2^{R(Q_f)}) / 2^{(c \cdot n^{3/4})}$ with at least 80% of the computed sizes meeting this condition and an aggregate mean circuit size less than or equal to $1.5 \cdot (\Omega(2^{R(Q_f)})) /$

$2^{(c \cdot n^{(3/4)})}$). The conjecture is refuted if any seed produces a circuit size that does not meet the lower bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	UNCERTAIN	1.00	HITS	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'quasi-quadratic forms' AND 'AC0 parity complexity' AND 'lower bound' - 'minimal rank' AND 'quadratic form' AND 'circuit size AC0' - 'degree less than or equal to n' AND 'quasi-quadratic forms' AND 'complexity theory'

Top relevant hits considered: - [http://arxiv.org/abs/cs/0301016v1] Lower Bounds on the Bounded Coefficient Complexity of Bilinear Maps - [http://arxiv.org/abs/2506.01645v2] The Price of Being Partial: Complexity of Partial Generalized Dominating Set on Bounded-Treewidth Graphs - [http://arxiv.org/abs/1401.4512v1] A quadratically tight partition bound for classical communication complexity and query complexity - [http://arxiv.org/abs/1904.00101v2] Stabilizer Circuits, Quadratic Forms, and Computing Matrix Rank - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/2506.13081v1] The Combinatorial Rank of Subsets: Metric Density in Finite Hamming Spaces - [http://arxiv.org/abs/2307.00917v5] A graph for which the second largest distance eigenvalue is less than $\frac{-3+\sqrt{5}}{2}$ is chordal - [http://arxiv.org/abs/0904.3478v1] Homotopy classification of nanophrases with less than or equal to four letters

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=1.5s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_quasi_quadratic_form(f):
        n = int(math.log2(len(f)))
        Q_f = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i+1, n):
```

```

        for x in range(2**n):
            if f[x ^ (1 << i)] != f[x ^ (1 << j)]:
                Q_f[i][j] += 1
                Q_f[j][i] += 1
    return Q_f

def matrix_rank(matrix):
    m, n = len(matrix), len(matrix[0])
    rank = 0
    for i in range(n):
        if all(matrix[j][i] == 0 for j in range(m)):
            continue
        rank += 1
        for j in range(i+1, n):
            if matrix[j][i] != 0:
                factor = matrix[j][i] / matrix[i][i]
                for k in range(n):
                    matrix[j][k] -= factor * matrix[i][k]
    return rank

def ac0_parity_circuit_size(f):
    n = int(math.log2(len(f)))
    circuit_size = 0
    for i in range(n):
        if f[1 << i] != f[0]:
            circuit_size += 1
    return circuit_size

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = generate_boolean_function(n)
    Q_f = compute_quasi_quadratic_form(f)
    R_Q_f = matrix_rank(Q_f)
    circuit_size = ac0_parity_circuit_size(f)

    if R_Q_f == 0 or circuit_size == 0:
        continue

    results.append({
        "n": n,
        "R_Q_f": R_Q_f,
        "circuit_size": circuit_size
    })

if not results:
    return {
        "metric_name": "AC0 Parity Circuit Size",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No valid instances found"
    }

mean_circuit_size = sum(result["circuit_size"] for result in results) / len(results)

```

```

support_fraction = sum(1 for result in results if result["circuit_size"] >= 2**(result["R_Q_f"]

return {
    "metric_name": "AC0 Parity Circuit Size",
    "metric_value": mean_circuit_size,
    "instances_tested": len(results),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": "" if support_fraction >= 0.8 else f"First failing seed with n={results[0]['n']}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_circuit_size = sum(result["metric_value"] for result in results if result["metric_value"] > 0) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_circuit_size} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"First failing seed\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=No valid instances found")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a241dfad.py", line 101, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a241dfad.py", line 63, in run_trial
    R_Q_f = matrix_rank(Q_f)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a241dfad.py", line 44, in matrix_rank
    factor = matrix[j][i] / matrix[i][i]
              ~~~~~^~~~~~
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error before producing data that could confirm or refute the conjecture. | next: Investigate and fix the division by zero error in the test code, then rerun the test with the pre-registered criteria to determine if the conjecture is supported or falsified.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13041
2	propose	ollama_remote	glm4:latest	0	0	12070
3	preregistration	ollama_remote	glm4:latest	0	0	7444
4	novelty	ollama_remote	glm4:latest	0	0	4968
5	novelty	ollama_remote	glm4:latest	0	0	8229
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42943
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11511
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9792
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12327
10	judge	ollama_remote	glm4:latest	0	0	23202

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 145528 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/6cd213d406ca.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/6cd213d406ca.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/6cd213d406ca.tar.gz (if generated)